L15: SLP to MLP

C. V. Jawahar

IIIT Hyderabad

March 9, 2026

Perceptron Learning Algorithm

Problem: Classify samples into two classes as $sign(\mathbf{w}^T\mathbf{x})$

$$w_i \leftarrow w_i + \Delta w_i$$

where

$$\Delta w_i = \eta(t - o)x_i$$

Where

- t = c(x) is target value.
- o is perceptron output.
- η is small constant called *learning rate*

Initialize all weights w_i to random values While not all examples are correctly predicted

- DO FOR each training example x in D
  - Compute current output o(x)
  - FOR each $i = 1$ to n
    $w_i < -w_i + \eta(t - o)x_i$

SLP: Recap

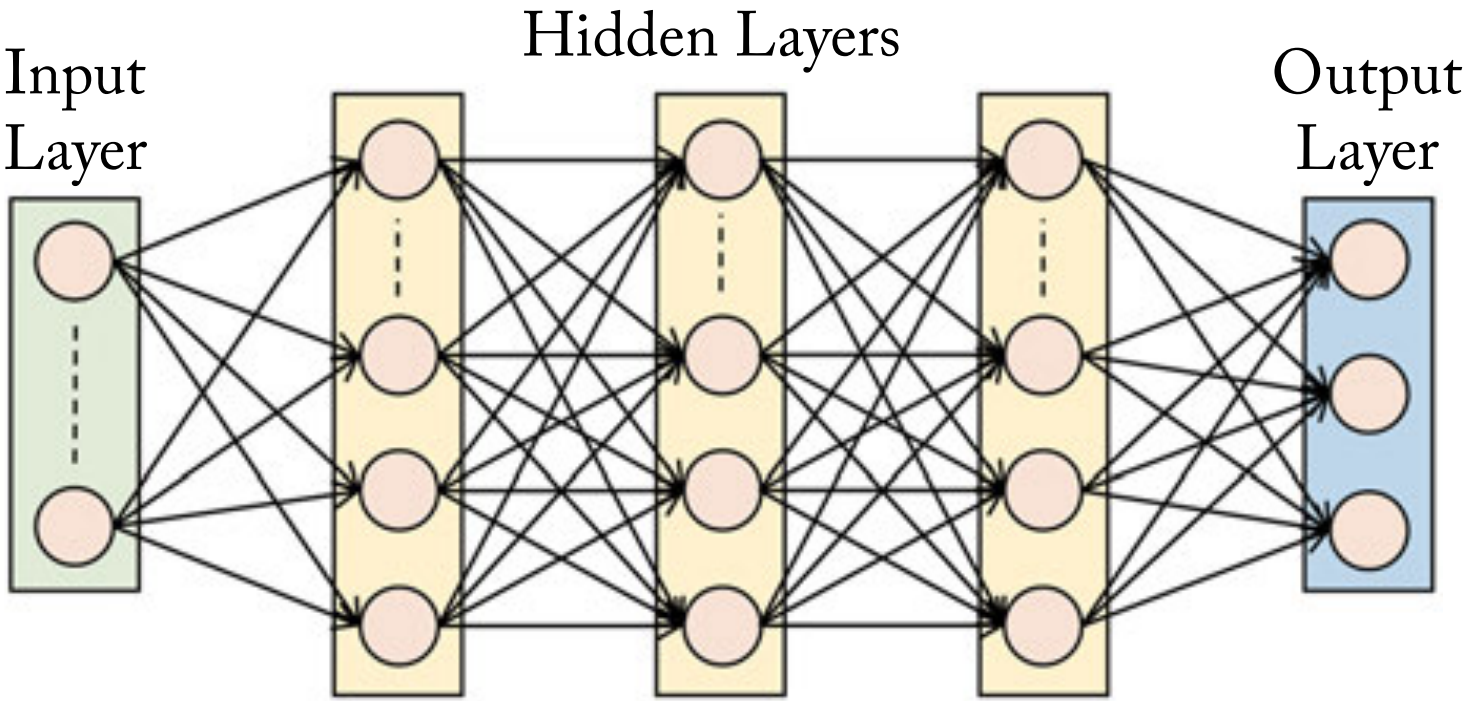
- Single Layer Perceptron Classifier
  - Converge when the data is linearly seperable
- Delta Rule (for regression)
  - Converge always

SLP: Recap

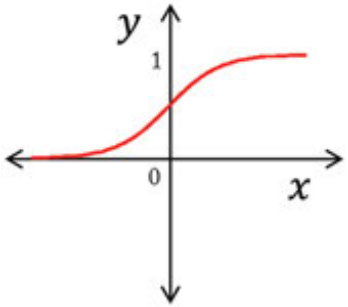
ExOR and Linear Non-Separability

- Problem
- Two layer solution
- (how did we solve with kernels?)
- (how do we solve it with Deep NN?)

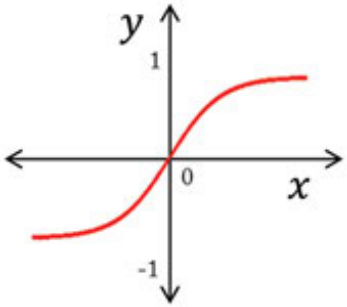
Multi-Layered Perceptron (MLP)



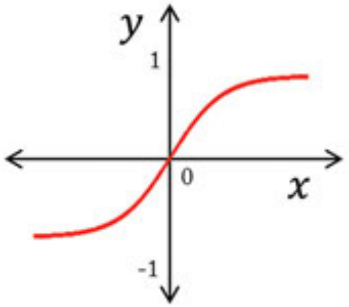
Activations



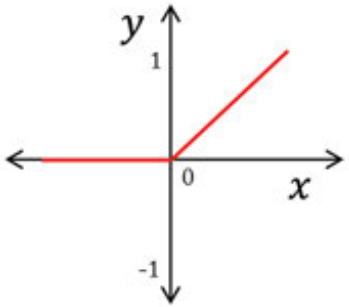
(a) Sigmoid



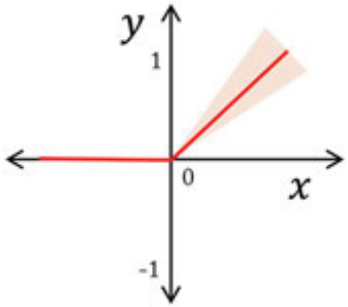
(b) Tanh



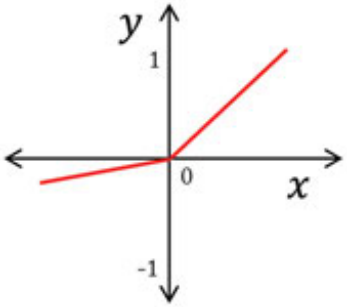
(c) Algebraic Sigmoid



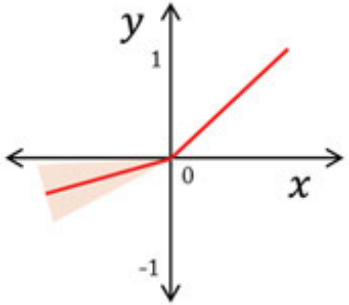
(d) ReLU



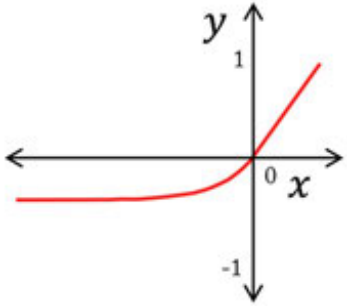
(e) Noisy ReLU



(f) Leaky ReLU/PReLU



(g) Randomized Leaky ReLU



(h) Exponential Linear Unit

Activations

1. **Sigmoid**

$$y = \frac{1}{1 + e^{-x}}$$

2. **tanh**

$$y = \frac{e^{x} + e^{-x}}{e^{x} + e^{x}}$$

3. **ReLu**

$$y = \max(0, x)$$

4. **Leakly Relu**

$$y = \begin{cases} x & \text{if } x > 0 \\ cx & \text{if } x \leq 0 \end{cases}$$

5. **Noisy ReLu**

$$y = \max(0, x + \epsilon); \epsilon = \mathcal{N}(0, \sigma(x))$$

6. **Exponential Linear Unit**

$$y = \begin{cases} x & \text{if } x > 0 \\ a(e^{x} - 1) & \text{if } x \leq 0 \end{cases}$$

Problem

Consider an MLP with three inputs, one hidden layer (i) of two neurons and (ii) sigmoid activations. Output layer has one neuron with linear activations. All the neurons in the hidden and output layers have bias.

- How many learnable weights are there?
- Write the function $y = f(w, x)$ explicitly with all the variables?
- Decompose represent as multiple linear and nonlinear functions

Derivatives

- **sigmoid**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$

- **tanh:** similarly prove it as $1 - \tanh^2(z)$

Question

- Draw networks that can solve AND, OR, EXOR with 0/1 logic. (assume neurons have a threshold activation. $\phi(x) = 1$ if $x \geq 0$)

Discussion Points

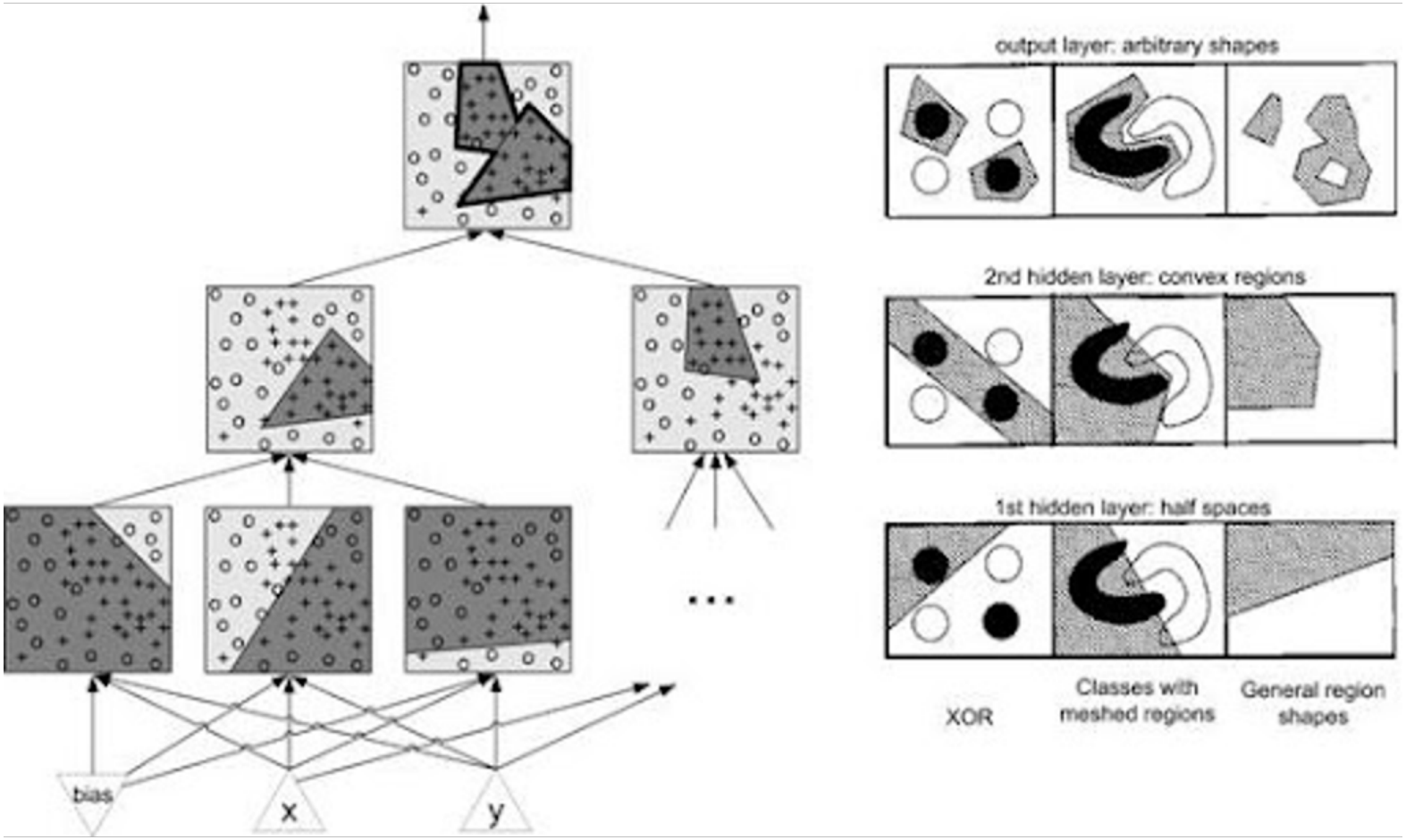
- **Q1: Why do we need hidden layers?**
- **Q2: Why do we need activation functions (nonlinearities)?**
- **Q3: What is the role of hidden layers?** (Hint: computing and enriching features or new representations.)

MLP

A typical MLP has

- **Nodes and Edges** Network has nodes (where simple computations take place and edges where information flow happen.
- **Layered architecture.** Network is understood and represented in layers.
- **Dense Connections** Successive layers are often fully connected.
- Not very deep
- Used as classifier and also as regressor with handcrafted small features.
- MLP often expends features before classification.

Multi-Layered Perceptron (MLP)



Multi-Layered Perceptron (MLP)

The Universal Approximation Theorem (UAT) for Multi-Layer Perceptrons (MLPs) states that a feedforward neural network with at least one hidden layer, a finite number of neurons, and a non-polynomial (non-linear) activation function can approximate any continuous function on a compact (closed and bounded) input space to arbitrary precision

MLP for Regression

- In many cases, the expected output need not be in $[0, 1]$ or $[-1, +1]$. It is advisable to use a linear activation ($\phi(x) = x$) in such cases.

MLP for Classification

- Avoid Argmax in the output
- Add a softmax layer that converts activations to "probabilities" as softmax: $\frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$
- Use cross entropy loss:

$$\mathcal{L}(\mathbf{o}, \mathbf{t}) = \sum_{i=1}^{K} t_i \log(o_i)$$

If $\mathbf{t}$ and $\mathbf{o}$ are probability distributions (desired and output), then the cross entropy loss is equivalent to the KL divergence [1] between these two distributions.

[1] https://en.wikipedia.org/wiki/Kullback-Leibler_divergence

Basic/Naive Back Propagation

1. Initialize the network with random weights.
2. For all the samples, compute the output of the neural network $\mathbf{x}_{p+1}$
3. Compute the loss for the full batch of N samples as

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} L(\mathbf{x}^i_{p+1}, \mathbf{y}_i)$$

4. Adjust all the parameters (such as weight matrices) as

$$\theta^{k+1} \leftarrow \theta^k - \Delta\theta$$

or

$$\theta^{k+1} \leftarrow \theta^k - \eta \frac{\partial L}{\partial \theta}$$

5. Repeat steps 2-4 (until ??).

Discussions on Training

Challenges in Training NNs

- Non-Convex Optimization
- Many parameters to Control

Why simple GD works?

- Second order methods (that needs Hessian matrix) is compute intensive
- Nature of Loss Function (simultaneously all gradients vanish?)
- Multiple effective solutions

Problem

Consider an MLP with three inputs, one hidden layer (i) of two neurons and (ii) sigmoid activations. Output layer has one neuron with linear activations. All the neurons in the hidden and output layers have bias. Can you differentiate Loss wrt to each of the learnable parameters (weights)? (use chain rule)